

Benemérita Universidad Autónoma de Puebla
Facultad de Ciencias de la Computación

Técnicas de inteligencia artificial

Reporte: Proyecto final



BUAP

Docente: Abraham Sánchez López

Alumno

Taisen Romero Bañuelos

Matrícula

202055209

Solving the 0-1 Knapsack Problem with Genetic Algorithms

El artículo habla sobre el problema de la mochila y cómo contrastan los resultados si se usa una selección por grupos o una ruleta para elegir las mejores soluciones para que sus genes pasen a la siguiente generación. A manera de expandir la investigación del artículo decidí aumentar el número de ítems que se le pueden agregar a la mochila, ya que usar sólo tres ítems para las pruebas puede resultar en un problema un poco trivial (de hecho, en el artículo se menciona que para aumentar la complejidad hay que aumentar el número de ítems). Naturalmente traté de seguir un enfoque realista para resolver este problema, pero creo que quedarse en un marco teórico no es suficiente para demostrar si de verdad son útiles los AG en un problema de la mochila para un caso de la vida real. Me basaré en una mochila de gimnasio para que el AG decida qué cosas tengo que llevar, y con base en mi experiencia, observaré si verdaderamente es adecuada la solución obtenida.

Para resolver este problema partí desde la recuperación del código usado en la actividad “Algoritmo genético para resolver el problema de la mochila”. Me basé en este código para replicar la resolución comentada en el artículo. Luego hice una experimentación para explorar la efectividad del AG ante un mayor número de artículos. Naturalmente, al basarse en la práctica anteriormente mencionada es que usé el paquete GA, lo cual en cierta forma hace que mi implementación no sea del todo fiel a lo que se expone en el artículo. Por eso, hice una tercera versión manual para que pueda replicar al 100% los pasos seguidos en el artículo (a excepción de la selección por ruleta, que en el artículo mismo se menciona que no es tan efectiva, por lo que no considero necesario replicar).

Primer acercamiento

Para refrescar la memoria mencionaré algunos aspectos que se replican del artículo. Se parte de una instancia sencilla con tres ítems. La función de aptitud penaliza proporcionalmente aquellas soluciones cuyo peso supera la capacidad de la mochila (13).

Nota: la función fitness está fuertemente basada en el código de la actividad “Algoritmo genético para resolver el problema de la mochila”.

```

> summary(gann_mod)
— Genetic Algorithm —

GA settings:
Type           = binary
Population size = 200
Number of generations = 300
Elitism        = 2
Crossover probability = 0.85
Mutation probability = 0.001

GA results:
Iterations      = 50
Fitness function value = 7
Solution =
  x1 x2 x3
[1,] 1 1 0

```

Figura 1. Resumen del primer AG.

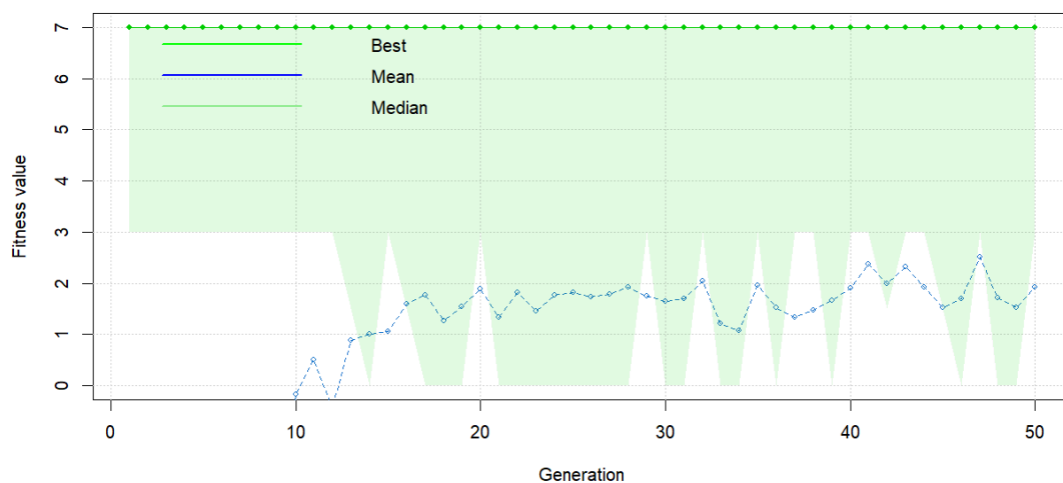


Figura 2. Evolución del primer AG.

Como vemos, alcanzamos el resultado óptimo en apenas 50 generaciones. Además, la mejor solución se mantiene estable desde el inicio, pero la convergencia global de la población es más lenta.

Si analizamos el siguiente plot notaremos que hay fuertes oscilaciones en el fitness, con caídas abruptas (valores negativos). Esto puede indicar que se generaron muchas soluciones inviables con gran penalización, lo que demuestra que la restricción de peso fue una buena idea para no considerar todos estos valores inviables.

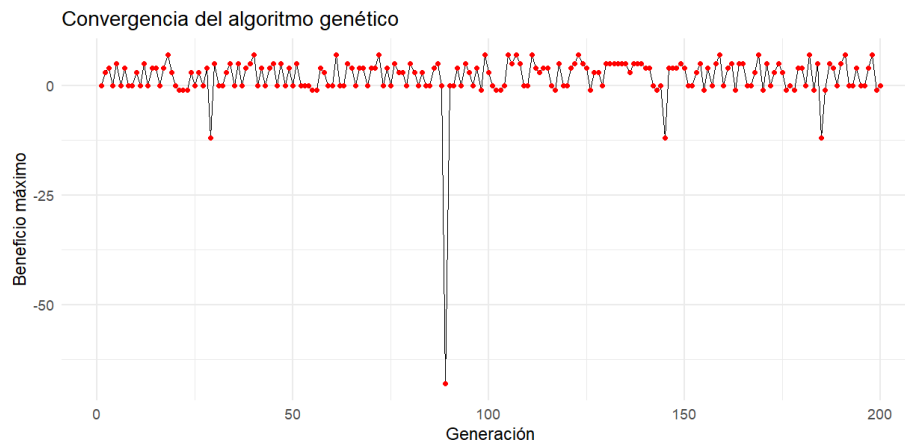


Figura 3. Evolución de la convergencia del primer AG.

Si extraemos la mejor solución podemos ver algo que será costumbre para los siguientes ejemplos. El límite de peso fue alcanzado, lo que indica que el AG aprovecha al máximo cada espacio de la mochila. Esto a primera vista es útil, pero yo me pregunto si en un caso de la vida real sería realmente útil exprimir al máximo cada rincón de la mochila. A veces llevar una carga menos es provechoso para la espalda o para poder agregar objetos nuevos no considerados, aunque supongo que esto se sale del contexto del problema planteado inicialmente. En fin, otro detalle interesante de este AG es que parece ser que le da una prioridad a la calidad (fitness) sobre la cantidad.

```
> print(df_sol)
# A tibble: 2 x 6
# Groups:   item, weight [2]
  item weight benefit freq total_weight total_benefit
  <chr>   <dbl>   <dbl> <int>      <dbl>      <dbl>
1 A         6         4     1         6         4
2 B         7         3     1         7         3
> cat("Beneficio total:", sum(df_sol$total_benefit), "\n")
Beneficio total: 7
> cat("Peso total:", sum(df_sol$total_weight), "\n")
Peso total: 13
```

Figura 4. Segundo resumen del primer AG.

¿Y si aumentamos el número de ítems?

Para analizar más a fondo la propuesta anterior aumenté el número de ítems a 10. Lo que hice para hacer todavía más interesante esta propuesta fue asignarle el peso y el beneficio a los ítems de manera aleatoria dentro de un rango numérico, con la diferencia de que configuré el peso para que la mayoría de esos números aleatorios sean bajos, algunos término medio y otros pocos con valores altos (con el objetivo de hacer más rica la combinatoria). Otro detalle a mencionar es que puse como valor fijo el límite de peso en 20 (ya que si voy a aumentar 7 ítems entonces le aumentaré 7

unidades al problema). La otra opción que consideré para establecer el límite de peso es basarme en el 40% del peso total de los pesos, pero no apliqué este enfoque porque la capacidad de una mochila depende de su diseño preestablecido y no sobre qué cosas me gustaría poner en una mochila (en todo caso el problema se trataría de buscar el tipo de mochila adecuada y no en la combinación de objetos adecuada).

En fin, al parecer el código nos generó una combinación balanceada de pesos y también de fitness, lo cual es bueno.

```
> print(df_item_long)
```

	item	weight	benefit
1	A	15	91
2	B	6	22
3	C	9	99
4	D	10	95
5	E	9	17
6	F	7	56
7	G	16	45
8	H	7	92
9	I	20	50
10	J	7	85

Figura 5. Datos de los 10 objetos a considerar.

Observando los resultados del AG podemos notar que el incremento de objetos apenas afectó al número de iteraciones en que se alcanza la combinación óptima. En parte tiene sentido porque sólo agregamos 7 objetos, pero no por ello es menos interesante. Otra cosa a destacar es que es más notoria esa decisión del AG por priorizar la calidad por sobre la cantidad, ya que si observamos la solución notaremos que apenas hay 3 ítems (justo los que tienen un mayor beneficio).

```
> summary(gann_mod_ext)
```

Genetic Algorithm

GA settings:

Type	=	binary
Population size	=	200
Number of generations	=	100
Elitism	=	2
Crossover probability	=	0.85
Mutation probability	=	0.001

GA results:

Iterations	=	52
Fitness function value	=	246

Solution =

	x1	x2	x3	x4	x5	x6	x7	x8	x9	x10
[1,]	0	0	1	0	0	0	0	1	0	1

Figura 6. Resumen del segundo AG.

```

> print(df_sol)
# A tibble: 3 × 6
  item weight benefit freq total_weight total_benefit
<chr> <dbl> <dbl> <int> <dbl> <dbl>
1 C      9      99      1      9      99
2 H      7      92      1      7      92
3 J      7      85      1      7      85
> df_sol$total_weight
[1] 9 7 7
> df_sol$total_benefit
[1] 99 92 85

```

Figura 7. Segundo resumen del segundo AG.

Otra buena señal de la efectividad del algoritmo es que mantiene una curva de aprendizaje positiva, ya que el porcentaje de soluciones válidas por generación muestra una tendencia ascendente. Aunque esta tendencia no es tan pronunciada como en otras configuraciones, refleja un proceso evolutivo estable, donde cada generación aporta nuevas combinaciones factibles. Esto indica que el algoritmo, pese a generar inicialmente muchas soluciones inviables, es capaz de adaptarse progresivamente a las restricciones impuestas.

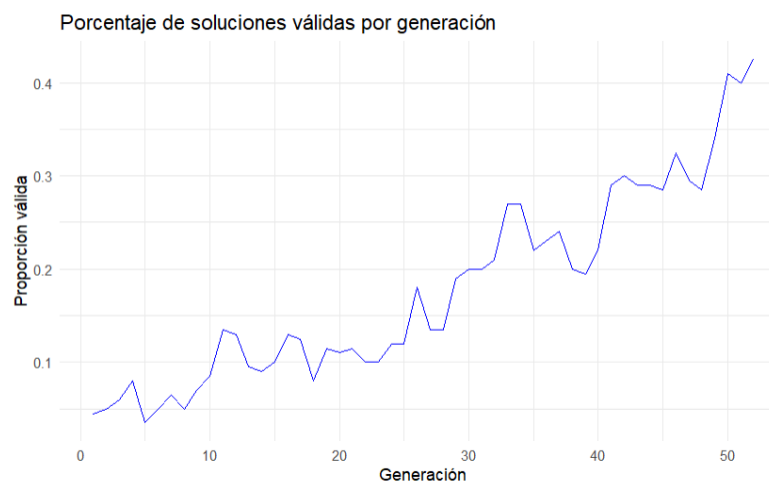


Figura 8. Porcentaje de soluciones válidas del segundo AG.

Una versión más fiel

Bien, como se mencionó al inicio del documento, las versiones anteriores del AG están fuertemente relacionadas al paquete GA, y es por esta razón que no aplicamos el método de selección que se aplica en el artículo (ya que el paquete GA no implementa group selection ni ruleta, sino selección proporcional). Aunque era consciente de esta limitación desde el principio, opté por incluir estas versiones iniciales con el objetivo de explorar diferentes enfoques y promover la diversidad de soluciones.

En cualquier caso, hay que trabajar en una nueva versión más fiel al artículo. Esta versión incluye **group selection** con probabilidades 50%-30%-15%-5% para favorecer la reproducción de los individuos más aptos, un **elitismo ajustado** (seleccionando explícitamente los mejores cromosomas y conservándolos en la nueva generación), y una **condición de convergencia** poblacional basada en el porcentaje de individuos con el mismo valor de aptitud (el AG se detiene cuando el 90% de la población presenta el mismo nivel de beneficio).

Gracias a que esta implementación es manual puede agregar y afinar el AG de manera más personalizada, prueba de ello es lo que se mencionó anteriormente. Lo podemos ver reflejado nuevamente en este mensaje que nos indica que la convergencia se alcanzó en la generación 11.

```
+ population <- new_pop
+ }
Convergencia alcanzada en generación 11
```

Figura 9. Aviso de convergencia alcanzada.

Siendo sincero los resultados me provocaron un sabor agridulce, ya que por un lado el que esta versión de la misma solución final que la implementación anterior me da a entender que es igual de efectiva, pese a que yo quería ver una respuesta. Quizá el sabor agridulce se me quite si analizo los análisis de rendimiento del AG, ya que, ¿realmente fue igual de efectivo este AG que el anterior?

```
> print(selected_items)
  item weight benefit
3     C      9      99
8     H      7      92
10    J      7      85
> total_weight
[1] 23
> total_benefit
[1] 276
```

Figura 10. Resultados del tercer AG.

La respuesta corta a la pregunta anterior es que no, esta nueva versión aprendió más rápido, probablemente debido a los ajustes realizados en su conjunto pero más al ajuste del elitismo. Me explico, si hacemos una copia como la que se hace en el apartado de elitismo es hasta cierto punto normal que alcancemos antes las soluciones con mayor beneficio.

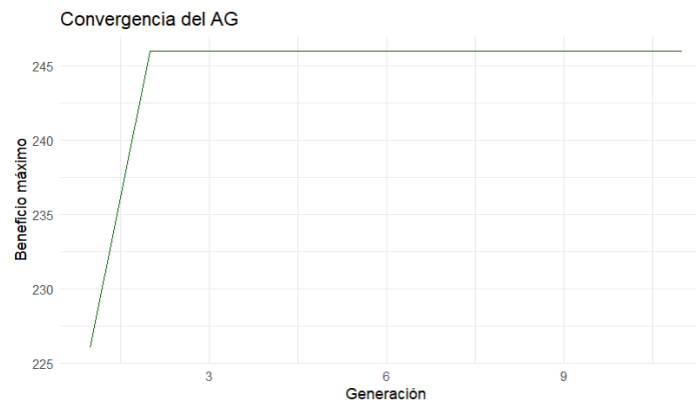


Figura 11. Observe lo rápido que alcanzó la convergencia óptima.

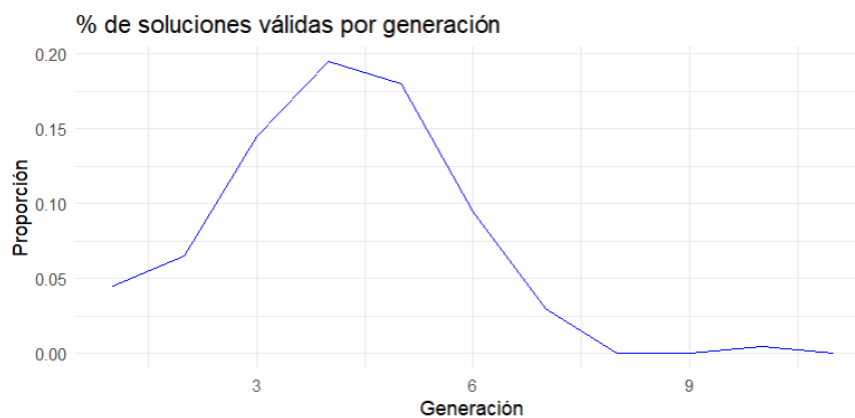


Figura 12. Porcentaje de soluciones válidas del tercer AG.

De hecho, el notar este mejor aprendizaje del nuevo AG me hizo reinterpretar el desempeño del AG anterior, ya que en este nuevo gráfico el bajo porcentaje de soluciones válidas no implica necesariamente un mal rendimiento, sino que podría deberse a que el algoritmo alcanzó rápidamente la solución óptima y continuó explorando regiones menos favorables.

Esta nueva implementación demostró la importancia de experimentar con diferentes propuestas a un mismo problema, ya que me permitió identificar factores clave que influyen en el desempeño (como el elitismo), y también me facilitó una comprensión más profunda de la dinámica del problema y del comportamiento de los AG bajo diferentes condiciones.

Un caso de la vida real

Para poner a prueba los AG con un caso de la vida real me basé en un dilema en el que estuve para decidir qué cosas eran fundamentales para mis entrenamientos (y para mi espalda). Para asignar el peso me basé en el espacio que ocupaba el objeto en mi mochila y para asignar el beneficio me basé en lo útil que resultaban dichos ítems

durante el entrenamiento. Para decidir el límite de peso me basé principalmente en una aproximación basada en lo que yo terminé llevando a mis entrenamientos, pero para que no limitara de forma artificial al AG a que de la misma respuesta que yo amplié ese límite un poco más con el fin de permitir más variabilidad. Cabe mencionar que hubiese sido igualmente interesante que para asignar los pesos me guié en sus dimensiones tridimensionales exactas, sin embargo, como se puede suponer esto representa varios inconvenientes (tanto para calcular el volumen de los objetos como el de la mochila), además, este factor no consideraría que algunos objetos pese a ser pequeños, pesan y ello influye en mi espalda (lo cual queremos evitar).

Bien, los pesos fueron los siguientes:

- Guantes de boxeo (peso: 8, beneficio: 10)
- Manoplas (peso: 9, beneficio: 5)
- Vendas (peso: 1, beneficio: 10)
- Bucal (peso: 1, beneficio: 9)
- Botas de boxeo (peso: 6, beneficio: 6)
- Botella de agua (peso: 4, beneficio: 10)
- Toalla (peso: 3, beneficio: 3)
- Vaselina (peso: 2, beneficio: 1)
- Careta (peso 8, beneficio 8)
- Desodorante (peso: 1, beneficio 8)
- Dulce (peso: 1, beneficio: 7)
- **Límite de peso: 16**

Los resultados fueron sorprendentes e hilarantes, pues, el AG me mostró exactamente la misma combinación de objetos que llevo en mi mochila para entrenar.

```
> print(selected_items)
      item weight benefit
1  Guantes de boxeo      8      10
3           Vendas       1      10
4           Bucal        1       9
6  Botella de agua       4      10
10          Desodorante   1       8
11           Dulce        1       7
```

Figura 13. Datos de los nuevos objetos.

Otro aspecto interesante es que en esta ocasión el AG tardó más generaciones en alcanzar la convergencia (17), lo que demuestra que el ejemplo anterior (convergencia en 11 generaciones) era algo trivial para el estudio.

```

+   population <- new_pop
+ }
Convergencia alcanzada en generación 17

```

Figura 14. Aviso de convergencia.

Si bien los resultados fueron iguales a los que mi conocimiento empírico concluyó quisiera pensar que con alguna semilla específica podríamos encontrar una combinación diferente (cabe mencionar que esto supone otro ejercicio de combinatoria).

En fin, el peso total fue de 16, lo que incita a pensar que se aprovechó al máximo el espacio disponible (pues incluir un dulce es omisible en ocasiones).

```

> total_weight
[1] 16
> total_benefit
[1] 54

```

Figura 15. Peso y beneficio de la solución óptima encontrada.

Si analizamos la proporción de individuos de la población que cumplen con la restricción de peso (≤ 16) en cada generación podemos notar que el AG aprendió a respetar la restricción de forma rápida, se ve en la gráfica que se estabiliza desde la generación 5 con más del 80% de soluciones viables. Esto sugiere que el mecanismo de penalización y selección funciona correctamente.

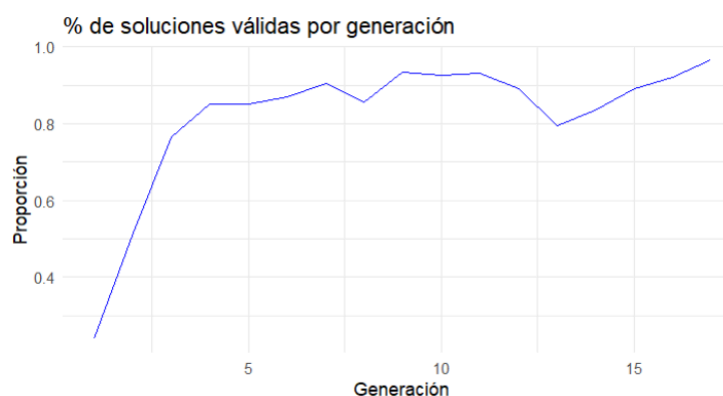


Figura 16. Porcentaje de soluciones válidas.

Y sobre la evolución de la convergencia no hay mucho que decir más que converge al beneficio máximo de 54 en apenas 6 generaciones y lo mantiene hasta el final. Esto sugiere que se ha encontrado una solución difícil de mejorar, lo cual es bueno en un problema con espacio de búsqueda acotado.

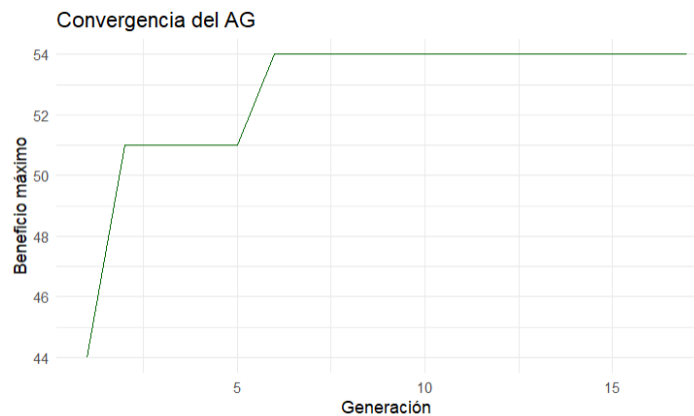


Figura 17. Convergencia en el ejemplo realista.

Conclusión

Los resultados obtenidos en este trabajo permiten extraer varias interpretaciones relevantes sobre el comportamiento y utilidad de los algoritmos genéticos en el contexto del problema de la mochila. En primer lugar, se observa que el AG tiende a priorizar la calidad de las soluciones por encima de la cantidad de objetos seleccionados, lo cual puede interpretarse como una inclinación natural hacia configuraciones eficientes y no saturadas. Si bien esto podría darse a entender como miel sobre hojuelas debemos cuestionar su aplicabilidad real, ya que en este contexto no se considera la versatilidad o el espacio libre restante. En el ejemplo realista que hice el resultado fue muy bueno, pero, ¿esto fue por qué adapté un problema para que encaje en este contexto, o porque el AG realmente fue capaz de modelar una decisión racional en un escenario práctico? En efecto, aunque el algoritmo encontró exactamente la misma combinación que yo suelo llevar a mis entrenamientos, esto puede deberse tanto a la estructura del problema como a los pesos y beneficios asignados desde mi experiencia.

Otro hallazgo importante es que una alta proporción de soluciones válidas no siempre es sinónimo de un mejor rendimiento. La disminución del porcentaje de soluciones válidas no necesariamente refleja un deterioro, sino una intensificación de la búsqueda en zonas menos prometedoras del espacio. En este sentido, el comportamiento del AG debería interpretarse más allá de métricas puntuales, pues, su éxito también radica en su capacidad de adaptación.